

**UNIVERSITY OF ENERGY AND NATURAL RESOURCES,
SUNYANI**

SCHOOL OF SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATICS

**AN INTEGRATED EXAMINATION TIMETABLING AND
INVIGILATOR VERIFICATION SYSTEM USING
CONSTRAINT-BASED SCHEDULING AND VENUE-BOUND
QR-CODE AUTHENTICATION**

BY

[STUDENT NAME]

[INDEX NUMBER]

A project report submitted to the Department of Computer Science and Informatics,
University of Energy and Natural Resources, in partial fulfilment of the requirements for
the award of the degree of Bachelor of Science in Computer Science.

SUPERVISOR: [SUPERVISOR NAME]

SEPTEMBER, 2026

DECLARATION

I hereby declare that this project report is the result of my own original work carried out under supervision, and that no part of it has been presented for another degree in this university or elsewhere. All sources of information used have been duly acknowledged by means of references.

Candidate's Signature: Date:

Name: [STUDENT NAME]

CERTIFICATION

I hereby certify that this project report was prepared by the above-named candidate under my supervision in accordance with the guidelines on project work laid down by the University of Energy and Natural Resources.

Supervisor's Signature: Date:

Name: [SUPERVISOR NAME]

ABSTRACT

Examination administration in many higher-education institutions remains a largely manual exercise: course lists are collected by electronic mail, examination timetables are constructed by hand in spreadsheets, and invigilator attendance is recorded on paper sign-in sheets that can be forged, signed remotely, or completed after the fact. This project designed, implemented and evaluated an integrated, web-based examination timetabling and invigilator verification system for the University of Energy and Natural Resources (UENR). The system digitises the complete examination lifecycle across six functional domains: academic structure management, structured course submission and approval, constraint-based examination timetable generation with venue allocation, automatic invigilator-to-venue assignment, QR-code-based physical presence verification, and real-time attendance monitoring. Timetables are generated by a greedy first-fit-decreasing heuristic with a three-pass constraint-relaxation strategy and up to twenty randomised retries, enforcing hard constraints on venue capacity and student-cohort clashes. Invigilators are assigned by a round-robin algorithm that prevents double-booking and same-department invigilation. Attendance is verified through venue-bound QR codes that encode the venue and examination session; a two-stage preview-then-confirm scan protocol validates the token format, the invigilator's assignment, the venue identity, duplicate attempts and a strict time window, persisting every outcome for forensic audit. A Socket.IO layer delivers real-time check-in and absence notifications to examination officers, supported by a background job that automatically marks missed scans as absent within five minutes. The system was implemented as a React 18 progressive web application over a Node.js/Express REST API with a PostgreSQL database accessed through Prisma ORM, and secured with JWT authentication, role-based access control, schema validation and immutable audit logging. Functional evaluation against the six research objectives confirmed clash-free timetables, conflict-free assignments, correct rejection of all fraudulent scan scenarios, and sub-five-minute absence detection. The work demonstrates that an integrated, venue-bound verification approach materially strengthens examination integrity without specialised hardware.

Keywords: examination timetabling, constraint-based scheduling, QR-code verification, invigilator attendance, role-based access control, real-time systems, progressive web application.

TABLE OF CONTENTS

CHAPTER ONE: INTRODUCTION	10
1.1 Background of the Study	10
1.2 Problem Statement	11
1.3 Aim of the Study	11
1.4 Specific Objectives	12
1.5 Research Questions	12
1.6 Significance of the Study	13
1.7 Scope of the Study	13
1.8 Limitations of the Study.....	14
1.9 Organisation of the Study	15
CHAPTER TWO: LITERATURE REVIEW	16
2.1 Introduction.....	16
2.2 The Examination Timetabling Problem.....	16
2.3 Automated Timetable Generation Approaches.....	16
2.4 Scheduling Constraints	18
2.5 Invigilator Allocation.....	18
2.6 Attendance Verification and Its Fraud Vectors	19
2.7 QR-Code Technology and Verification Applications.....	19
2.8 Real-Time Web Communication	20
2.9 Authentication, Authorisation and Audit Concepts	20
2.10 Review of Existing Systems	21
2.11 Research Gap	22
2.12 Chapter Summary	22
CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN	23
3.1 Research Design.....	23
3.2 Development Methodology	23

3.3 Requirements Gathering	24
3.4 Requirements Analysis	24
3.4.1 Functional Requirements	24
3.4.2 Non-Functional Requirements	25
3.5 System Analysis.....	26
3.5.1 Analysis of the Existing System	26
3.5.2 Actors and Use Cases.....	27
3.6 System Architecture.....	27
3.7 Database Design.....	29
3.8 Algorithm Design.....	31
3.8.1 Timetable Generation.....	31
3.8.2 Invigilator Assignment.....	31
3.8.3 Scan Evaluation	32
3.9 Technology Selection.....	32
3.10 Testing and Evaluation Methodology	33
CHAPTER FOUR: SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION .	35
4.1 Implementation Overview	35
4.2 User Roles and Access Control.....	36
4.3 Objective 1: Course Submission and Approval Workflow.....	36
4.3.1 Implementation	36
4.3.2 Results.....	37
4.4 Objective 2: Constraint-Based Timetable Generation	37
4.4.1 Implementation	37
4.4.2 Results.....	39
4.5 Objective 3: QR-Code-Based Presence Verification.....	40
4.5.1 Implementation	40
4.5.2 Results.....	42

4.6 Objective 4: Automatic Invigilator Assignment	42
4.6.1 Implementation	42
4.6.2 Results.....	44
4.7 Objective 5: Real-Time Notification and Automatic Absence Detection	44
4.7.1 Implementation	44
4.7.2 Results.....	45
4.8 Objective 6: Role-Based Access Control and Audit Logging	45
4.8.1 Implementation	45
4.8.2 Results.....	46
4.9 Security Implementation Summary	46
4.10 Discussion	47
CHAPTER FIVE: CONCLUSION, RECOMMENDATIONS AND FUTURE WORK ..	49
5.1 Conclusion	49
5.2 Recommendations.....	50
5.3 Future Work.....	50
5.4 Closing Remarks.....	51
REFERENCES	53

LIST OF FIGURES

Figure ? : Use case diagram of the Timetabling and Invigilator Verification System	27
Figure ? : Three-tier system architecture	28
Figure ? : API request lifecycle through the security middleware pipeline.....	29
Figure ? : Simplified entity-relationship diagram of the fifteen Prisma models.....	30
Figure ? : End-to-end examination lifecycle supported by the system	35
Figure ? : Course approval workflow state machine	37
Figure ? : Constraint-based timetable generation algorithm flow	39
Figure ? : Two-stage QR-code verification flow with the server-side validation pipeline.	41
Figure ? : Automatic invigilator assignment algorithm	43
Figure ? : Real-time notification architecture with role-scoped rooms and the auto-absent job	45

LIST OF TABLES

Table 1: Stakeholders and the significance of the system to each	13
Table 2: Comparison of automated examination timetabling approaches	17
Table 3: User-bound versus venue-bound QR verification strategies	19
Table 4: Capability comparison of existing system categories and the present work	21
Table 5: Iterative development phases and their deliverables	23
Table 6: Functional requirements of the system	24
Table 7: Non-functional requirements of the system	25
Table 8: Comparison of the existing manual process with the proposed system.....	26
Table 9: Database models by functional group	29
Table 10: Technology stack and selection rationale	32
Table 11: Evaluation criteria and methods per objective	33
Table 12: Role-permission matrix.....	36
Table 13: Timetable generation constraint verification results.....	40
Table 14: QR verification scenario test results	42
Table 15: Real-time event catalogue.....	44
Table 16: Security measures by layer	46

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
ERD	Entity-Relationship Diagram
GPS	Global Positioning System
HTTPS	Hypertext Transfer Protocol Secure
JWT	JSON Web Token
NP-hard	Non-deterministic Polynomial-time hard
ORM	Object-Relational Mapping
PWA	Progressive Web Application
QR	Quick Response (code)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
TLS	Transport Layer Security
UENR	University of Energy and Natural Resources
UI	User Interface

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Every semester, universities across the world go through the same demanding ritual: organising end-of-semester examinations. It sounds straightforward on paper, but in practice it is one of the most logistically heavy tasks in academic administration. At the University of Energy and Natural Resources (UENR), a single examination period can involve several departments, dozens of venues, hundreds of courses spread across different levels, and a pool of invigilators — all squeezed into a few weeks. The examination office has to make sure every approved course gets a slot and a venue without clashing with other papers the same students will write, that each venue has enough invigilators, and that those invigilators actually show up where they are supposed to, when they are supposed to.

For the most part, this has been handled manually. Heads of department e-mail their course lists to the examination office, often in formats that differ from one department to the next. Someone then types these lists into a spreadsheet, checks for duplicates by eye, and builds the timetable by hand — moving courses around cells until nothing obviously clashes. Invigilators get printed notices telling them where to go, and on the day of the exam they sign a paper sheet at the venue. It works, but only just, and it breaks down quickly as the number of courses and venues grows.

The research community has long known that examination timetabling is an NP-hard problem [1], [2]. What this means in plain terms is that no algorithm can guarantee a perfect timetable for a realistic-sized university in a reasonable time, and doing it by hand is even less practical. Over the past few years, researchers have continued to explore heuristics, metaheuristics and hybrid approaches to close that gap [3], [4], [5]. On the attendance side, paper sign-in sheets have well-documented weaknesses: they can be forged, signed on someone's behalf, or filled in after the fact [12]. QR-code-based attendance systems have gained traction as a lighter alternative, since most staff already carry a smartphone capable of scanning a code [10], [11].

What has changed recently is that the building blocks for a fully integrated solution — responsive single-page web applications, progressive web app (PWA) installation [16], real-time push communication [24], and mature server-side frameworks — are now

reliable enough to combine into one system. This project brings those pieces together to digitise the entire examination lifecycle at UENR, from course submission through timetable generation to QR-code-based verification of invigilator presence, with real-time monitoring throughout.

1.2 Problem Statement

At UENR the examination office runs the whole lifecycle manually. Heads of department e-mail course lists; the officer transcribes them, checks for duplicates, and tracks approval status through back-and-forth messages. Once courses are approved, the officer constructs the timetable by hand, visually scanning for clashes. Invigilators are assigned through printed schedules, and they confirm attendance by signing a paper sheet at the venue.

This setup has several problems that keep recurring:

1. Paper sign-in sheets can be forged or signed for an absent colleague. Nothing in the current process actually proves the invigilator was at the right venue at the right time.
2. Invigilator no-shows are discovered too late — usually only when students complain about an unsupervised hall, or when the sheets are collected days later.
3. There is no audit trail. Decisions about course approvals, timetable changes and invigilator assignments are not formally logged, so accountability is hard to establish after the fact.
4. Communication is slow. Invigilators may get their assignment details late, and the examination office has no live picture of what is happening across venues.

Taken together, these gaps expose the university to scheduling clashes, undetected absenteeism and examination malpractice, while piling administrative work onto the examination office. A systematic, automated and verifiable approach is needed.

1.3 Aim of the Study

The aim of this project is to design and implement a web-based examination timetabling and invigilator verification system that automates constraint-based examination scheduling, uses QR codes to verify that invigilators are physically present at their assigned venues, and gives examination officers real-time visibility of attendance at the University of Energy and Natural Resources.

1.4 Specific Objectives

To achieve this aim, the study pursues six specific objectives:

5. Develop a structured course submission and approval workflow that replaces e-mail-based collection with a validated, auditable digital process.
6. Implement a constraint-based automatic timetable generation algorithm that schedules approved courses into slots and venues while preventing clashes and respecting venue capacities.
7. Design and implement a QR-code-based verification mechanism that confirms an invigilator's physical presence at the correct venue during the assigned time window.
8. Implement automatic invigilator-to-venue assignment with conflict-avoidance rules that prevent double-booking and same-department invigilation.
9. Develop a real-time notification subsystem that instantly informs examination officers of check-ins, automatically detected absences and pending account approvals.
10. Enforce role-based access control and immutable audit logging across all system operations to preserve examination integrity and accountability.

1.5 Research Questions

Each objective maps to a research question that guides the study:

11. How can a structured digital workflow replace e-mail-based course collection to ensure validated, auditable course submission and approval?
12. What constraint-based scheduling algorithm can automatically generate clash-free examination timetables with venue allocation at the scale of a mid-sized university?
13. How can QR-code technology be used to reliably confirm an invigilator's physical presence at the correct venue during the assigned time window?
14. What rules and algorithms can automatically assign invigilators to venues while avoiding scheduling conflicts and same-department bias?
15. How can real-time communication improve the examination office's ability to monitor invigilator attendance and respond promptly to absences?
16. What access control and audit mechanisms are needed to maintain examination integrity and accountability in a multi-role administrative system?

1.6 Significance of the Study

The significance of the study can be looked at from the perspective of each group of stakeholders involved, as summarised in Table 2.

Table 1: Stakeholders and the significance of the system to each

Stakeholder	Role in the Examination Process	Benefit Derived from the System
Examination Officer (Super Admin)	Manages the entire examination lifecycle	Automated timetabling, real-time visibility of invigilator attendance, immutable audit trail, reduced administrative burden
Head of Department	Submits and manages departmental courses	Structured, validated course submission with immediate approval feedback and status tracking
Invigilator	Supervises examinations at assigned venues	Clear and timely assignment notifications, transparent and tamper-proof attendance verification
Students (indirect)	Sit the examinations	Clash-free schedules and adequately supervised examination halls
University administration (indirect)	Accountable for academic integrity	Reduced malpractice risk and demonstrable accountability for administrative actions
Future researchers	Study examination management systems	A documented reference implementation combining constraint-based scheduling with venue-bound QR verification

Beyond the immediate stakeholders, the study makes a practical contribution to the literature by showing that venue-bound (rather than user-bound) QR tokens, combined with server-side validation and a background absence detector, can close the fraud vectors inherent in paper-based invigilator attendance — and they can do it without biometric hardware.

1.7 Scope of the Study

The study covers the design, implementation and functional evaluation of a multi-user web application for UENR. It includes:

- Users: three roles — Super Admin (examination officer), Head of Department and Invigilator — with self-registration, e-mail verification and administrative approval.
- Processes: academic structure management; course submission and approval; examination session and venue management; constraint-based timetable generation; automatic and manual invigilator assignment; venue QR-code generation; two-stage attendance scanning; real-time notification; automatic absence detection; and audit logging.
- Technologies: React 18 single-page application delivered as a progressive web application; Node.js/Express REST API; PostgreSQL database accessed through Prisma ORM; and Socket.IO for real-time communication.

The study does not cover student-facing functionality (students do not log in), invigilator swap and replacement workflows, bulk course import, analytics dashboards, native mobile apps, or web push notifications. These are revisited as future work in Chapter Five.

1.8 Limitations of the Study

The following limitations should be kept in mind when reading the results in this report:

17. Verification of the system was done manually through the deployed application; no automated unit, integration or end-to-end test suite was developed.
18. The greedy scheduling heuristic guarantees feasibility with respect to the hard constraints but does not guarantee an optimal timetable.
19. Audit log entries are recorded in the database, but a dedicated in-application viewer for the audit trail was not completed.
20. Offline capability of the progressive web application is limited to caching of the application shell; scans cannot be queued offline.
21. GPS-based location verification is optional — invigilators may decline the browser location permission, in which case off-campus scanning is logged with a warning rather than blocked.
22. The system is deployed in a single cloud region, which adds avoidable network latency for users in Ghana.
23. Formal user-acceptance testing with UENR examination office staff had not been conducted at the time of writing.

1.9 Organisation of the Study

The rest of this report is organised as follows. Chapter Two reviews the literature on examination timetabling, scheduling algorithms, invigilator allocation, attendance verification, QR-code technology, real-time web communication and the security concepts behind multi-role web systems, and identifies the research gap this work addresses. Chapter Three presents the research design, development methodology, requirements analysis, system architecture, database design, algorithm design, technology selection and evaluation criteria. Chapter Four describes the implementation, presents the results of evaluating each objective, and discusses the findings. Chapter Five concludes the study, offers recommendations to the university, and outlines directions for future work.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter looks at the body of work relevant to the six areas this project touches on: the examination timetabling problem and how hard it is; the algorithms people have used to tackle it; the constraints involved; how invigilators get allocated; how attendance is verified and where QR codes fit in; real-time web communication; and the authentication, authorisation and audit ideas that underpin multi-role systems. After reviewing existing systems, the chapter identifies the gap that motivates this project.

2.2 The Examination Timetabling Problem

Examination timetabling is the task of assigning a set of exams to a limited number of time slots and rooms while satisfying a set of constraints. It has been studied for decades and is well established as an NP-hard problem [1], [2]. In practical terms, this means there is no known algorithm that can guarantee the best possible timetable for a realistic-sized university in a reasonable amount of time, so researchers and practitioners rely on heuristics and metaheuristics instead.

Recent surveys have kept the picture up to date. Siew et al. reviewed solution methodologies for exam timetabling from 2012 to 2023, covering mathematical optimisation, heuristics, metaheuristics, hyper-heuristics and hybrid approaches [1]. Ceschia et al. surveyed educational timetabling formulations and benchmarks, noting that real-world instances routinely involve room capacities, room splitting for large cohorts and institution-specific rules that simplified academic benchmarks leave out [2]. Both observations matter directly to this project, where large service courses must be split across several venues and where UENR's own rules (for example, a cohort writing at most one exam per day) must be honoured.

2.3 Automated Timetable Generation Approaches

A wide range of techniques has been applied to examination timetabling. At one end are constructive greedy heuristics — first-fit and first-fit-decreasing orderings — where exams are sorted by some measure of difficulty (usually enrolment size or constraint density) and placed one at a time into the first feasible slot. These are simple, fast, and

produce workable solutions quickly, especially when combined with randomised restarts and multi-pass constraint relaxation [6], [7]. At the other end are metaheuristics — simulated annealing, tabu search, genetic algorithms — which iteratively improve an initial solution and generally achieve better quality at the cost of more computation and complexity [3], [4].

Abdullah and Hassan compared a constructive heuristic against a genetic algorithm on a real-world dataset and found that the constructive heuristic actually outperformed the genetic algorithm in resolving all student conflicts, while producing feasible schedules in seconds [8]. Al-Betar et al. reviewed optimisation techniques in university timetabling and noted that metaheuristics dominate the recent literature, but that greedy constructive methods remain widely used in practice for their speed and simplicity [4]. Table 3 summarises the trade-offs.

Table 2: Comparison of automated examination timetabling approaches

Approach	Representative work	Strengths	Weaknesses
Greedy / first-fit-decreasing with restarts	Muklason et al. [6]; Abdullah & Hassan [8]	Very fast; simple; predictable; easy to extend with institution-specific rules	No optimality guarantee; quality depends on ordering and randomisation
Constraint programming (CP)	Carlsson et al. [3]	Declarative constraints; can prove infeasibility	Modelling effort; solver runtime grows sharply with instance size
Metaheuristics (SA, tabu, GA)	Muklason et al. [7]; Al-Betar et al. [4]	High solution quality; good soft-constraint satisfaction	Long runtimes; many parameters to tune; complex implementation
Hybrid / matheuristic	Siew et al. [1]	Combines speed of heuristics with quality of metaheuristics	More complex to design and maintain

The system implemented in this project uses a greedy first-fit-decreasing constructor with three fallback passes of progressively relaxed constraints and up to twenty randomised retries, keeping the best result found. This sits squarely in the practical tradition: for the scale of UENR's course portfolio (tens to low hundreds of courses per semester), the

heuristic finishes in seconds while satisfying all hard constraints, and the retry mechanism recovers much of the quality that more expensive search techniques would provide.

2.4 Scheduling Constraints

Constraints in examination timetabling are usually split into hard constraints (which must be satisfied for a timetable to be valid) and soft constraints (whose violation degrades quality but not validity) [1], [2]. Common hard constraints include: no student sits two exams at the same time; room capacity is not exceeded; every exam is scheduled exactly once. Common soft constraints include spreading a cohort's exams across the period, avoiding consecutive exams for the same cohort, and scheduling large exams early to allow marking time [2].

Because UENR does not maintain individual student enrolment data within the examination office, the system approximates the student-clash constraint at the cohort level: no two exams for the same department and academic level may share a slot, and a department-level cohort may write at most one exam per day. This cohort-level approximation is a recognised strategy where individual enrolment data is not available [2].

2.5 Invigilator Allocation

Invigilator (proctor) allocation is a related but separate scheduling problem: given a fixed timetable, assign invigilators to venue sessions subject to availability, workload-fairness and conflict constraints. Cimen et al. addressed the invigilator assignment problem using a mixed-integer linear programming model and a heuristic algorithm, considering real-life concerns such as fair distribution of workload, reduction of successive duties and prioritisation based on invigilators' profession [9]. Their work confirms that the problem is usually solved sequentially after the timetable is fixed, which mirrors administrative practice.

This project follows the sequential approach: the timetable is generated first, and invigilators are then assigned by a round-robin algorithm with conflict avoidance. Two constraints distinguish the present formulation: an invigilator may never be assigned to two venues in the same slot (no double-booking), and an invigilator may not supervise an exam offered by their own department — a fairness rule intended to reduce the opportunity for partiality that is seldom made explicit in the literature.

2.6 Attendance Verification and Its Fraud Vectors

Verifying that staff actually show up at examinations is operationally important but does not get much research attention. Paper-based sign-in sheets, the most common mechanism, are open to at least four kinds of fraud: forged signatures; proxy signing by colleagues; remote signing (signing without attending); and retrospective completion after the session has ended [12]. Biometric systems — fingerprint or face recognition — have been proposed for attendance tracking, but they need dedicated hardware at every venue, raise privacy concerns, and have been aimed almost entirely at student rather than staff attendance.

2.7 QR-Code Technology and Verification Applications

The Quick Response (QR) code is a two-dimensional matrix barcode with high data density, error correction and the ability to be decoded by standard smartphone cameras. In education, QR codes have been applied mostly to student attendance: a per-session code is displayed and students scan it to register presence [10], [11], [13]. The reported weaknesses of these schemes include code sharing (a student forwards the code image to absent peers) and the lack of any binding between the code and a physical location [12].

Nwabuwe et al. tackled these weaknesses directly by combining dynamic QR codes, geofencing and IMEI verification to mitigate proxy attendance, buddy-punching and early departure [12]. Their work shows that binding the QR code to a physical location — through geofencing — is one of the most effective ways to prevent attendance fraud. The system in this project takes a similar philosophical position but applies it to invigilator verification rather than student attendance, and does so without requiring a native mobile app.

The key design decision here is that QR codes are venue-bound rather than user-bound. Each code encodes the tuple (venue identifier, examination session identifier); it is printed and physically posted at the venue it names. An invigilator must therefore be physically present at the correct venue to scan the correct code, and scanning any other venue's code is detected server-side and rejected. Table 4 contrasts the two binding strategies.

Table 3: User-bound versus venue-bound QR verification strategies

Property	User-bound QR code	Venue-bound QR code (this project)
What the code identifies	The individual user (or session ticket)	The physical venue and examination session
Where the code lives	On the user's device; trivially forwarded	Printed and posted at the venue
Proof of physical presence	None — can be scanned from anywhere	Strong — scanning requires presence at the venue poster
Wrong-venue detection	Not possible	Automatic (server compares assignment with scanned venue)
Fraud vector: code sharing	High risk	Ineffective — the code is public but useless without a valid assignment and time window
Server-side validation required	Optional	Mandatory — assignment, duplicate and window checks

2.8 Real-Time Web Communication

Traditional HTTP request-response interaction requires the client to poll for new information, which wastes bandwidth and delays notification. The WebSocket protocol establishes a persistent, full-duplex channel over which a server can push events to connected clients with very low latency. Socket.IO is a widely used library layered above WebSocket that adds automatic reconnection, room-based broadcast semantics and a graceful fallback to HTTP long-polling where WebSocket connections cannot be established [24]. In examination management, real-time push is valuable for notifying the examination office of invigilator check-ins the moment they occur, alerting officers to detected absences, and propagating course-approval events to the affected department heads without a page refresh.

2.9 Authentication, Authorisation and Audit Concepts

Multi-role administrative systems need solid identity and access management. JSON Web Tokens (JWT) provide a compact, signed representation of authentication claims that allows stateless session validation on every request. Role-Based Access Control (RBAC) assigns permissions to roles rather than individuals, simplifying administration and

supporting the principle of least privilege. For credential storage, the bcrypt algorithm applies an adaptive, deliberately expensive key-derivation function that resists brute-force attack even as hardware improves; it remains a recommended choice, with a work factor of 10 or higher, alongside newer alternatives like Argon2id [20], [21]. Input validation, parameterised database queries and hardened HTTP response headers address the injection and configuration risks catalogued by the OWASP Top Ten [19]. An immutable audit log — an append-only record of who did what, to which object, when and from where — is a standard control for establishing accountability in administrative systems [19], [23].

2.10 Review of Existing Systems

Four categories of existing systems overlap with the scope of this project. Table 5 summarises their coverage against the capabilities the UENR problem context requires.

Table 4: Capability comparison of existing system categories and the present work

System category	Timetable generation	Invigilator assignment	Physical presence verification	Real-time monitoring	Audit trail
Commercial timetabling suites	Yes	Partial	No	No	Partial
QR student-attendance systems [10], [11]	No	No	Weak (user/session-bound codes)	No	Partial
Fraud-resistant QR + geofencing [12]	No	No	Strong (location-bound)	No	Partial
Learning management systems	No	No	No	No	Yes
This project	Yes (constraint-based)	Yes (conflict-free round-robin)	Strong (venue-bound QR, no extra hardware)	Yes (Socket.IO push + auto-absent)	Yes (immutable log)

Commercial scheduling suites focus on timetable construction and, in some cases, invigilator rostering, but none verifies that a rostered invigilator physically attended. QR attendance systems verify presence only weakly, because their codes are not bound to a location. Fraud-resistant systems like Nwabuwe et al.'s [12] verify presence strongly but focus on student attendance and do not integrate with timetabling. Many universities run bespoke in-house systems, but these are rarely published, which limits comparison [2].

2.11 Research Gap

The review shows that while the individual pieces — timetabling, invigilator allocation and attendance verification — have each been studied, no published system integrates the full examination lifecycle in a single cohesive application. Specifically:

24. Existing QR attendance research addresses student attendance with user- or session-bound codes; applying venue-bound QR codes to invigilator verification has not been explored.
25. Existing timetabling systems do not integrate with attendance verification, so a mismatch between the published roster and actual attendance goes undetected.
26. No reviewed system combines instantaneous check-in notification with automatic absence detection, leaving examination offices to discover absences reactively.
27. The accountability chain — from course approval through timetable change to venue scan — is not captured end-to-end in any reviewed system.

This project addresses the gap by building an integrated system that combines constraint-based timetable generation, conflict-free invigilator assignment, venue-bound QR-code verification with a two-stage scan protocol, real-time push notification and immutable audit logging, evaluated in the specific operational context of UENR.

2.12 Chapter Summary

This chapter placed the project within the literature on examination timetabling and attendance verification. The NP-hard nature of the scheduling problem justifies a heuristic approach; the fraud vectors of paper-based attendance justify a server-validated, venue-bound QR mechanism; and the absence of any integrated published system justifies the design developed in the next chapter.

CHAPTER THREE: METHODOLOGY AND SYSTEM DESIGN

3.1 Research Design

This project takes an applied research approach, carried out through the design science research methodology. Design science research is about creating and evaluating artefacts — constructs, models, methods and working systems — that address real organisational problems [17]. Storey et al. showed that design science is a well-suited lens for presenting and assessing software engineering contributions, even though it is underused in the field [17]. Baskerville et al. demonstrated its application to a specific information systems problem and offered practical guidance for researchers adopting the paradigm [18].

The present study follows the standard design science process: the problem was identified through observation of UENR's manual examination administration (Chapter One); solution objectives were derived as six specific objectives; the artefact — a full-stack web application — was designed and developed (this chapter and Chapter Four); it was demonstrated on live infrastructure; evaluated against explicit criteria for each objective (Section 3.10 and Chapter Four); and is communicated through this report.

3.2 Development Methodology

The system was built using an iterative and incremental approach. Development went through eight phases, each delivering a self-contained, testable increment that was exercised through the deployed application before the next phase began. Table 6 lists the phases and what each one delivered. This phasing is reflected directly in the modular structure of the backend codebase, where route modules are annotated with the phase that introduced them.

Table 5: Iterative development phases and their deliverables

Phase	Increment delivered	Principal modules
1	Project scaffolding, database provisioning, deployment pipeline	server bootstrap, Prisma schema, CI deployment
2	Authentication, registration windows, e-mail verification, account approval	auth, users, registrationWindows
3	Academic structure management	departments, academicYears, semesters

4	Course submission and approval workflow	courses, courseLevels
5	Examination sessions and invigilation records	examinationSessions, invigilations
6	Attendance scanning and venue assignments	attendance, venueAssignments
7	Audit logging and settings	auditLogs, settings
8	Venue management and timetable generation	venues, timetable

3.3 Requirements Gathering

System requirements were gathered from four sources: (i) direct observation of the existing manual process at the UENR examination office, including the artefacts it produces (e-mailed course lists, spreadsheet timetables and paper sign-in sheets); (ii) analysis of the failure modes of that process, as catalogued in the problem statement; (iii) domain knowledge of university examination administration practice; and (iv) the literature reviewed in Chapter Two, which supplied established constraint formulations and verification requirements.

3.4 Requirements Analysis

3.4.1 Functional Requirements

Table 6: Functional requirements of the system

ID	Functional requirement
FR1	Heads of Department shall create, edit and submit courses for approval.
FR2	Super Admins shall approve or reject submitted courses, optionally with comments.
FR3	Approved courses shall be locked against modification by Heads of Department.
FR4	Super Admins shall create examination sessions and manage venues and capacities.
FR5	The system shall automatically generate examination timetables using a constraint-based algorithm and report any unscheduled courses with reasons.
FR6	The system shall automatically assign invigilators to venues with conflict avoidance; manual assignment with equivalent checks shall also be supported.
FR7	Super Admins shall generate venue-specific QR codes, individually or in bulk per session.
FR8	Invigilators shall scan venue QR codes with their device camera to verify attendance.

FR9	The system shall validate every scan against QR format, invigilator status, venue assignment, duplicates and the permitted time window.
FR10	A two-stage scan protocol (preview, then confirm) shall prevent spurious database writes.
FR11	Real-time notifications shall be delivered to Super Admins for check-ins, absences, pending accounts and course submissions.
FR12	The system shall automatically mark invigilators absent when no valid scan occurs within the examination window.
FR13	Role-based access control shall be enforced on every operation.
FR14	All privileged actions shall be recorded in an immutable audit log.
FR15	Users shall self-register during time-boxed registration windows with 6-digit e-mail verification.
FR16	Newly registered accounts shall require Super Admin approval before first sign-in.
FR17	Password reset shall be supported through e-mailed tokens.
FR18	The client shall be installable as a progressive web application.

3.4.2 Non-Functional Requirements

Table 7: Non-functional requirements of the system

ID	Non-functional requirement	Category
NFR1	The interface shall be responsive and usable on desktop and mobile devices.	Usability
NFR2	All production communication shall use HTTPS.	Security
NFR3	All database access shall use parameterised queries to prevent SQL injection.	Security
NFR4	Passwords shall be hashed with bcrypt at a configurable cost factor.	Security
NFR5	All inputs shall be validated against declared schemas (Zod) before processing.	Security / reliability
NFR6	Authentication endpoints shall be rate-limited to resist brute-force attacks.	Security
NFR7	HTTP responses shall carry hardened security headers (Helmet).	Security
NFR8	State changes shall propagate to connected clients without page refreshes.	Performance / usability

NFR9	The system shall be deployable on managed cloud platforms (Vercel, Render, Neon).	Portability
NFR10	Timetable generation shall complete within seconds for typical UENR loads.	Performance

3.5 System Analysis

3.5.1 Analysis of the Existing System

The existing examination administration process at UENR is entirely manual. Table 8 contrasts each of its stages with the corresponding stage of the proposed system.

Table 8: Comparison of the existing manual process with the proposed system

Lifecycle stage	Existing (manual) system	Proposed system
Course collection	E-mailed lists in inconsistent formats; manual re-entry	Structured forms with validation; unique [code, semester] enforcement
Approval tracking	Informal correspondence; no state model	Explicit DRAFT -> SUBMITTED -> APPROVED/REJECTED state machine with locking
Timetable construction	Hand-placed in spreadsheets; clashes checked visually	Greedy constraint-based generator; clash-free by construction; unscheduled report
Invigilator assignment	Printed notices; double-booking possible	Round-robin with conflict avoidance; e-mail and in-app notification
Attendance verification	Paper sign-in sheets; forgeable	Venue-bound QR scan with server-side validation and time windows
Absence detection	Reactive; discovered days later	Automatic within five minutes of the missed window
Accountability	None	Immutable audit log of all privileged actions

3.5.2 Actors and Use Cases

The system serves three actors. The Super Admin (examination officer) runs the whole lifecycle: account approval, academic structure, course approval, venues, sessions, timetable generation, invigilator assignment, QR-code management and attendance monitoring. The Head of Department manages the course portfolio of one department: creating, editing and submitting courses, configuring course levels and viewing the published timetable. The Invigilator views personal venue assignments, performs QR scans at examination time and reviews their own attendance history. Figure 3 presents the use case model.

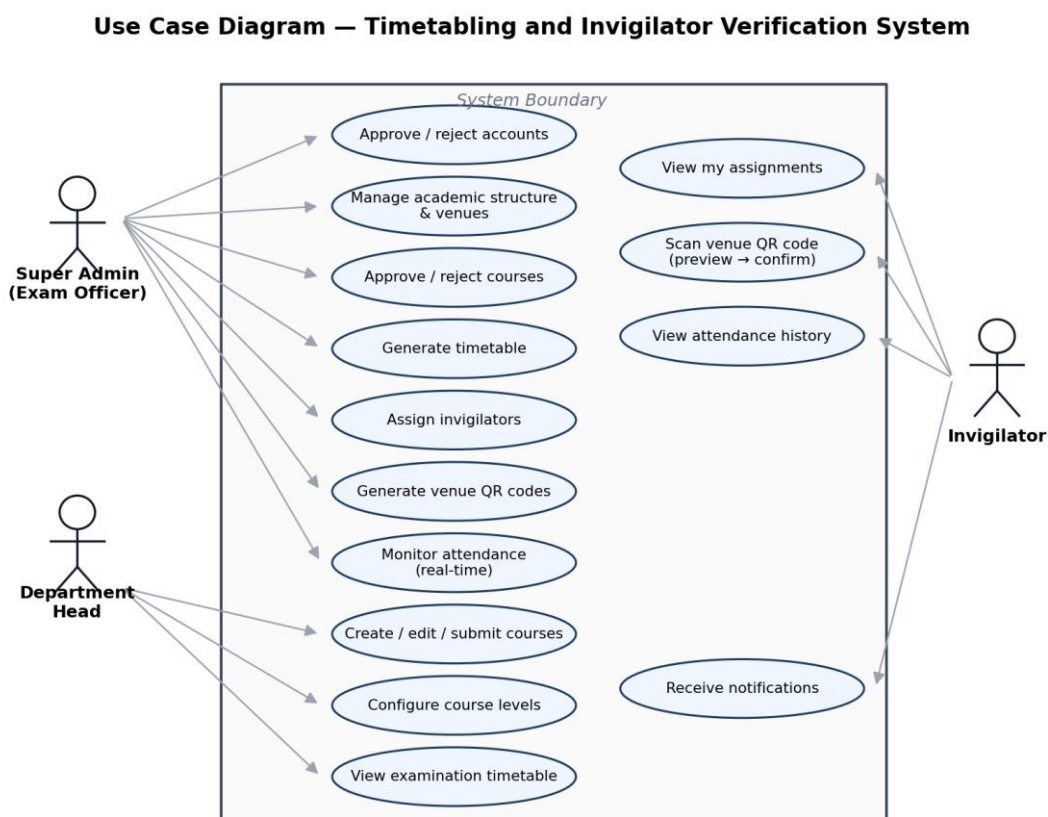


Figure 1: Use case diagram of the Timetabling and Invigilator Verification System

3.6 System Architecture

The system uses a three-tier client-server architecture, shown in Figure 1. The presentation tier is a React 18 single-page application [27], delivered as an installable progressive web application [16] and hosted on Vercel. The application tier is a Node.js 20 / Express 4 REST API hosted on Render, organised into sixteen feature modules that each follow a routes -> controller -> service layering; controllers are deliberately thin,

translating HTTP requests into calls on services that hold all the business rules, so the same services can be reused by the Socket.IO layer and background jobs. The data tier is a PostgreSQL 16 database hosted on Neon and accessed exclusively through Prisma ORM.

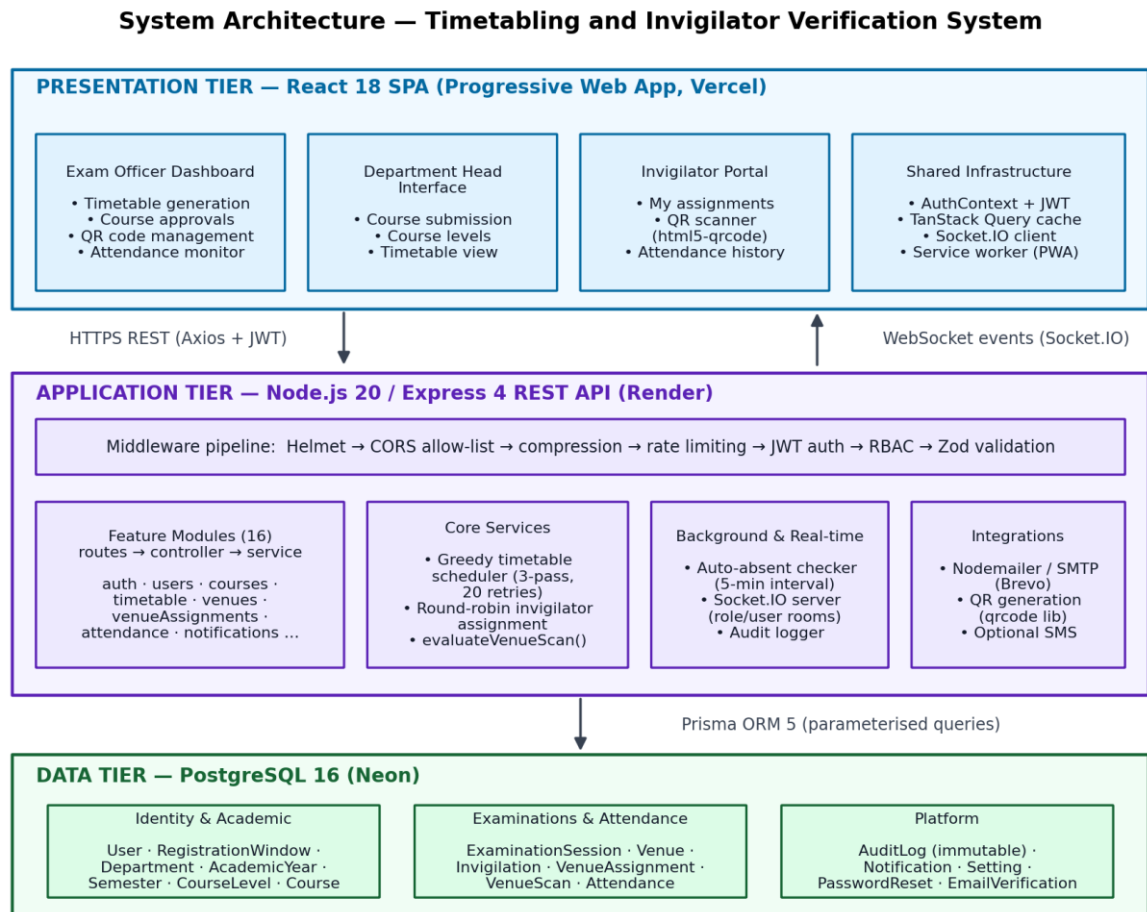


Figure 2: Three-tier system architecture

Every API request passes through a fixed middleware pipeline before reaching business logic, as shown in Figure 2: Helmet applies hardened response headers [23]; the CORS layer admits only the configured client origin; the rate limiter throttles abusive clients; requireAuth verifies the JWT and loads the requesting user (with a sixty-second cache) while rejecting non-active accounts; requireRole enforces the role list declared for the route; and a Zod schema [26] validates the request body, parameters and query string. Privileged operations then produce three side effects in addition to their primary result: an audit log entry, a persisted notification, and a Socket.IO broadcast to the affected rooms.

API Request Lifecycle — Security Middleware Pipeline

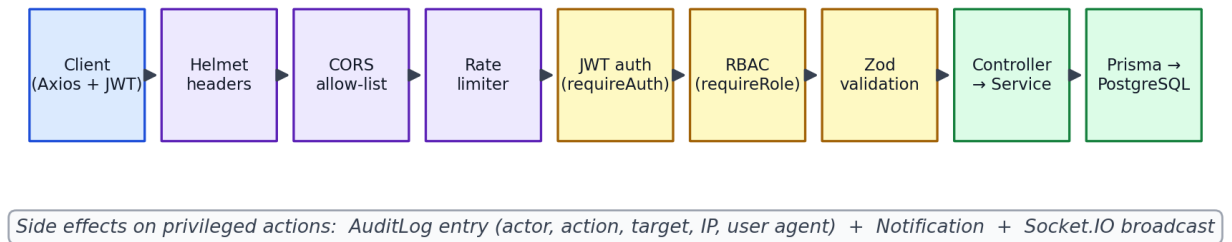


Figure 3: API request lifecycle through the security middleware pipeline

3.7 Database Design

The relational schema has fifteen models organised into six functional groups, with four enumerated types. Figure 4 presents the simplified entity-relationship diagram, and Table 9 summarises the grouping. Referential integrity follows two policies: children owned by a parent (for example, a department's courses) cascade on delete, whereas the audit log's actor reference is set to null on user deletion so that historical accountability survives account removal. Composite indexes are defined on the hot query paths, notably [examinationSessionId, venueId, slotAt] for assignment and scan lookups, and a unique constraint on [examinationSessionId, venueId, slotAt, invigilatorId] structurally prevents duplicate assignments.

Table 9: Database models by functional group

Group	Models	Purpose
Identity & access	User, RegistrationWindow, PasswordReset, EmailVerification	Accounts, roles, statuses, self-registration windows and credential recovery
Academic structure	Department, AcademicYear, Semester, CourseLevel	The institutional hierarchy into which courses are organised
Curriculum	Course	Courses with approval state, enrolment count, duration and locking
Examinations	ExaminationSession, Venue, Invigilation	Sessions, physical venues with capacities, and generated timetable

3.8 Algorithm Design

3.8.1 Timetable Generation

The generator is a greedy first-fit-decreasing constructor with multi-pass constraint relaxation and randomised restarts — an approach in the practical tradition identified by Muklason et al. [6] and Abdullah and Hassan [8]. Its pseudo-code is given below; the full flow appears in Chapter Four (Figure 6).

```
procedure GENERATE_TIMETABLE(session, dateRange, maxRetries = 20)
require: no SUBMITTED courses pending; at least 3 active venues
slots <- {08:00, 11:00, 14:00} x weekdays(dateRange)
groups <- group APPROVED courses by (code, title) // service
courses
sort groups: practical first, lower level first, larger enrolment first
best <- empty
for attempt in 1..maxRetries:
  shuffle(slots); sort venues by capacity ascending
  placement <- {}
  for pass in [FULL+GAP, FULL, RELAXED]:
    for each unplaced group g:
      for each slot s in slots:
        if HARD_OK(g, s, placement, pass) and VENUES_FIT(g, s):
          place g in s; allocate smallest fitting venues; split if
needed
          break
  remove any entries violating clash constraints // safety net
  if |placement| > |best|: best <- placement
  if all groups placed: break
persist best as Invigilation records; report unscheduled with reasons
```

HARD_OK enforces, in every pass, that venue capacity is respected, that no two exams of the same department and level share a slot, and that each course is scheduled exactly once. In passes one and two it additionally forbids more than one exam per day for a department-level cohort, and pass one further imposes a randomised rest gap (55% one day, 25% two days, 10% three days) between that cohort's consecutive exams. VENUES_FIT allocates the smallest venue whose capacity accommodates the course; where no single venue suffices, the course is split across several venues with sequential student ranges (for example, 1-150 and 151-300), and grouped service courses are placed in distinct venues of the same slot wherever possible.

3.8.2 Invigilator Assignment

The invigilator assignment follows the sequential approach recommended by Cimen et al. [9], with a round-robin heuristic for conflict avoidance:

```

procedure ASSIGN_INVIGILATORS(session)
  pool <- all ACTIVE invigilators; cursor <- 0
  for each (venue, slot) group in timetable(session):
    needed <- min(4, max(1, ceil(students / 50)))
    while assigned < needed:
      candidate <- pool[cursor mod |pool|]; advance cursor
      if candidate booked in slot -> skip
      if candidate.department in group's course departments -> skip
      if all candidates exhausted -> leave remainder unassigned;
    break
  persist VenueAssignment(session, venue, slot, candidate)
  notify each assigned invigilator (in-app + e-mail); emit socket events

```

The staffing ratio of one invigilator per fifty students, capped at four per venue and slot, reflects UENR practice. The algorithm deliberately prefers leaving a slot understaffed — which is surfaced to the officer for manual resolution — over ever double-booking an invigilator. Manual assignment through the interface passes through the same constraint checks, with two additions: an invigilator may hold only one time frame per day, and the per-venue maximum may not be exceeded.

3.8.3 Scan Evaluation

Both stages of the scan protocol call a single shared function, `evaluateVenueScan`, which runs the seven-step validation pipeline (QR format; invigilator role and status; venue and session existence; examination period; venue assignment for the current slot; duplicate detection; and the time window from fifteen minutes before slot start to thirty minutes after slot end). Centralising the pipeline in one function guarantees that the preview stage and the confirm stage can never diverge in their verdicts. The full flow is presented in Chapter Four (Figure 7).

3.9 Technology Selection

Table 10: Technology stack and selection rationale

Technology	Version	Justification
React	18	Component-based SPA framework with concurrent features and mature ecosystem [27]; enables the PWA delivery model
Vite	5	Fast development server and optimised production builds using native ES modules
TailwindCSS	3	Utility-first styling yielding consistent, responsive design without bespoke CSS
TanStack Query	5	Declarative server-state caching, retries and cache invalidation on

		socket events [25]
React Hook Form + Zod	7 / 3	Performant form handling with schema validation shared conceptually with the backend [26]
html5-qrcode / qrcode	-	Browser-native camera scanning via getUserMedia and server-side QR image generation, avoiding any native app
Node.js / Express	20 / 4	Single-language full stack; event-driven model well suited to I/O-bound API workloads
Prisma ORM	5	Type-safe data access, generated migrations and parameterised queries eliminating SQL injection
PostgreSQL	16	Mature relational engine with strong ACID guarantees, enums and composite constraints
Socket.IO	4	Room-based real-time broadcast with automatic WebSocket/long-polling fallback [24]
JWT (jsonwebtoken)	-	Stateless authentication suited to horizontally scalable APIs
bcryptjs	-	Adaptive password hashing resistant to brute force [20], [21]
Helmet, CORS, express-rate-limit	-	Defence-in-depth HTTP hardening aligned with OWASP guidance [19], [23]
Vercel / Render / Neon / Brevo	-	Managed hosting for client, API, database and transactional e-mail respectively

3.10 Testing and Evaluation Methodology

Verification of the system was done manually against the deployed application, using four complementary strategies: functional testing of every feature through the interface; negative testing of role boundaries by attempting operations as unauthorised roles, both through the interface and directly against the API; scenario testing of the scan pipeline covering every rejection class; and real-time testing with multiple concurrently connected browsers to confirm event delivery without refresh. Table 11 defines the evaluation criterion and method for each objective; the results are reported in Chapter Four.

Table 11: Evaluation criteria and methods per objective

Objective	Evaluation criteria	Method
1. Course workflow	All state transitions enforced server-side; locked courses immutable; audit entries	Exercise each transition; attempt to edit locked course; inspect audit

	created	records
2. Timetable generation	No clashes in output; venue capacity respected; unscheduled courses reported with reasons	Generate timetables over seeded data; systematically inspect all entries
3. QR verification	Every fraudulent scenario rejected with the correct code; valid scans recorded with metadata	Perform valid, wrong-venue, duplicate, out-of-window and malformed scans
4. Invigilator assignment	No double-booking; no same-department invigilation; staffing ratio respected	Run automatic assignment; inspect all resulting assignments
5. Real-time notification	Events received by connected officers without refresh; ABSENT records created within 5 minutes	Concurrent-browser observation; allow a scan window to lapse
6. RBAC and audit	Unauthorised requests rejected with 403; every privileged action logged	Cross-role API probing; audit table inspection

CHAPTER FOUR: SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION

4.1 Implementation Overview

The system was built as a monorepo with two directories: a client folder holding the React 18 progressive web application [16], [27], built with Vite and deployed on Vercel, and a server folder holding the Node.js/Express API, deployed on Render, with the PostgreSQL database hosted on Neon. The production deployment is publicly reachable at <https://unertimetable.vercel.app>. The backend consists of sixteen feature modules, each contributing routes, a controller, a service and a Zod validation schema [26]; the frontend has more than thirty route screens organised by domain, guarded by a ProtectedRoute component and served through role-filtered navigation. Figure 5 shows where the system's features sit within the end-to-end examination lifecycle it automates.

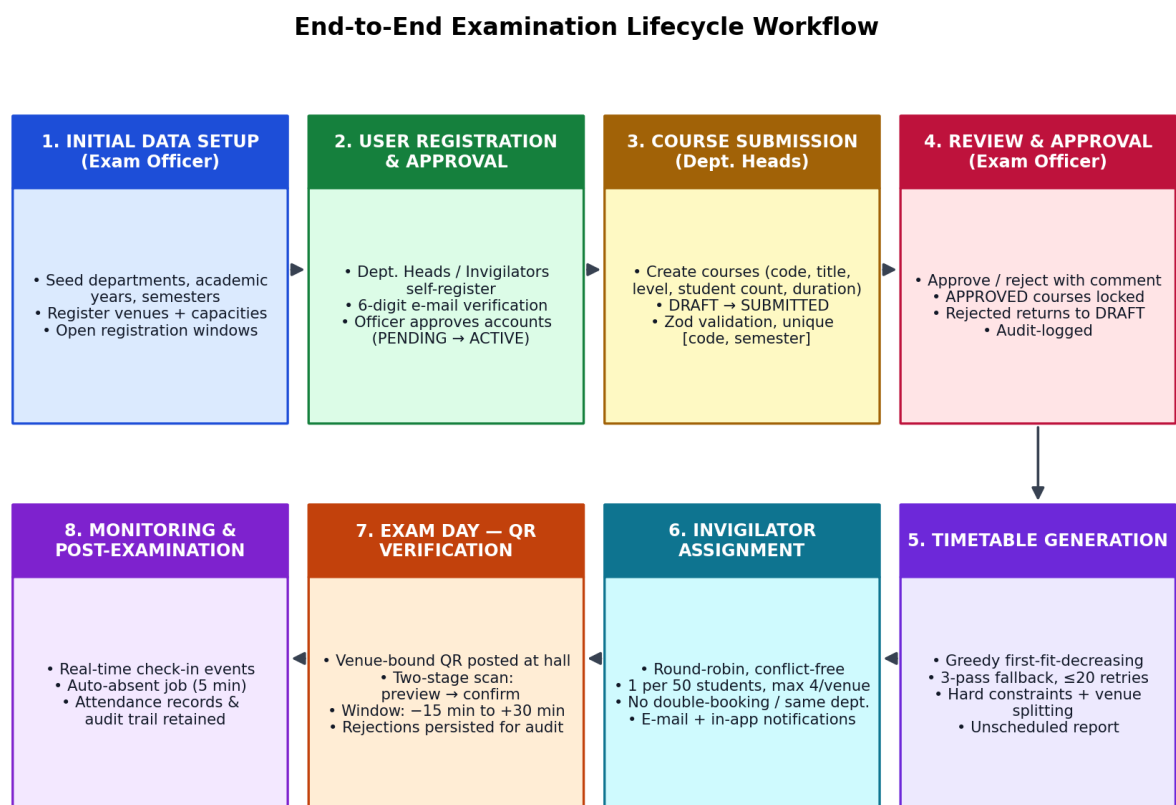


Figure 5: End-to-end examination lifecycle supported by the system

4.2 User Roles and Access Control

Three roles partition the system's capabilities, enforced independently at the API layer (requireAuth and requireRole middleware) and the interface layer (ProtectedRoute and role-filtered navigation). Table 12 summarises the permission matrix.

Table 12: Role-permission matrix

Capability	Super Admin	Dept. Head	Invigilator
Approve or reject user accounts	Yes	No	No
Manage departments, years, semesters, venues	Yes	No	No
Create, edit and submit courses	No	Yes (own department)	No
Approve or reject courses	Yes	No	No
Create examination sessions	Yes	No	No
Generate and edit timetable	Yes	View only	View own
Assign invigilators (auto and manual)	Yes	No	No
Generate venue QR codes	Yes	No	No
Scan venue QR code	No	No	Yes (own assignment)
View attendance records	Yes (all)	No	Yes (own history)
Receive real-time operational events	Yes	Course events only	Own assignments only

4.3 Objective 1: Course Submission and Approval Workflow

4.3.1 Implementation

The courses module implements a server-enforced state machine, shown in Figure 8. Heads of Department create courses with code, title, level, credit hours, student count, examination duration, special requirements, instructor and a practical-course flag; each course is scoped to the head's department and a selected semester, and uniqueness of the pair [code, semesterId] is enforced by the database. Courses begin in the DRAFT state, in

which they are freely editable. Submission moves a course to SUBMITTED and notifies all Super Admins in real time. Approval sets APPROVED, records the approver and timestamp, and sets locked = true, after which the course is immutable to the department; rejection records an optional comment and returns the course to DRAFT for revision. Every transition writes an audit entry capturing actor, action, target, result and request metadata.

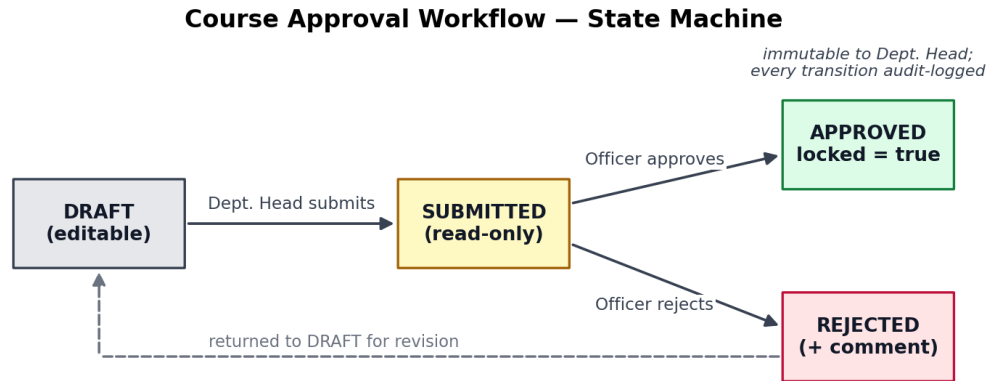


Figure 6: Course approval workflow state machine

4.3.2 Results

Functional evaluation confirmed that all transitions are enforced server-side and cannot be bypassed by direct API calls: attempts to edit a locked course as a Head of Department were rejected, attempts to approve a course as a non-admin returned 403 Forbidden, and audit entries were present for every submit, approve and reject action performed during testing. The workflow therefore fully replaces e-mail-based collection with a validated, auditable process, satisfying Objective 1.

4.4 Objective 2: Constraint-Based Timetable Generation

4.4.1 Implementation

The timetable service implements the algorithm designed in Section 3.8.1, following the greedy constructive tradition validated by Muklason et al. [6] and Abdullah and Hassan [8]. Figure 6 shows the complete flow. Generation is refused until every submitted course has been resolved and at least three active venues exist — preconditions that surface data-completeness problems before scheduling rather than after. Candidate slots comprise

three daily periods (08:00, 11:00 and 14:00) across the weekdays of the selected range, with weekends skipped by default. Service courses sharing a code and title across departments are grouped and placed atomically in a single slot, normally in different venues, so that a common paper is written simultaneously by all cohorts. The three-pass relaxation strategy, randomised restarts (up to twenty, retaining the best placement found), best-fit venue allocation with course splitting, and a post-placement clash safety net complete the pipeline. Persisted entries carry the scan window boundaries (`windowOpensAt` at slot start; `windowClosesAt` thirty minutes after slot end) later consumed by the attendance subsystem.

Constraint-Based Timetable Generation — Algorithm Flow

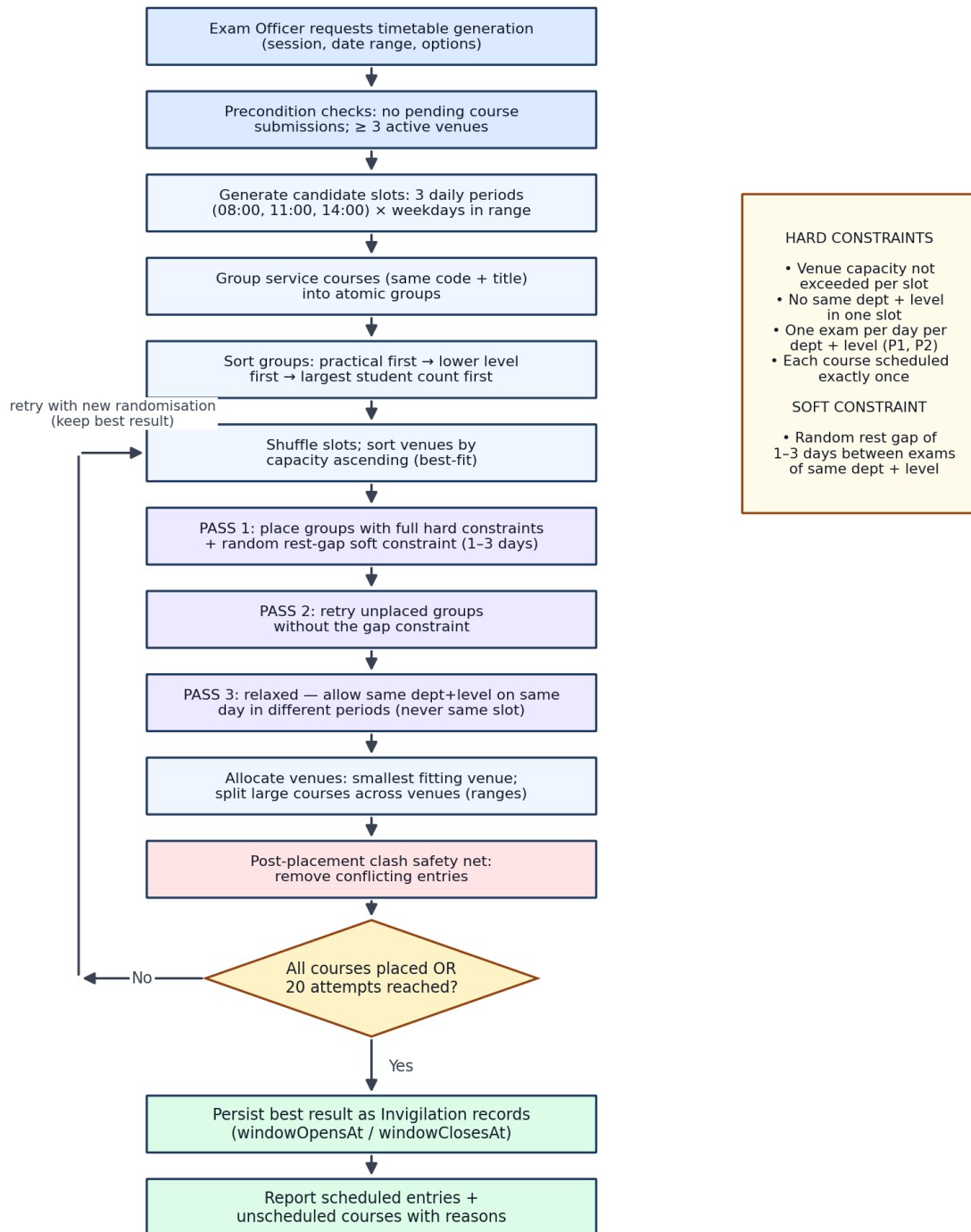


Figure 7: Constraint-based timetable generation algorithm flow

4.4.2 Results

Table 13 summarises the constraint verification performed on generated timetables over the seeded UENR-scale data set. In every generation run the hard constraints held for all

placed entries; where the period was deliberately made too short, the generator degraded gracefully, reporting the unplaced courses with actionable reasons such as 'student count exceeds every venue capacity' or 'no conflict-free slot available in the selected period'. Generation completed in seconds for course loads in the low hundreds. The greedy approach does not guarantee optimal spread — an accepted trade-off discussed in Section 4.10 — but the randomised retry strategy consistently recovered complete placements where feasible ones existed, consistent with the findings of Abdullah and Hassan [8].

Table 13: Timetable generation constraint verification results

Constraint verified	Type	Outcome
No two exams of the same department and level in one slot	Hard	Held in all runs
At most one exam per day per department-level cohort (passes 1-2)	Hard	Held; relaxed only in pass 3, never within a slot
Student count within allocated venue capacity	Hard	Held in all runs; large courses split with sequential ranges
Each approved course scheduled exactly once	Hard	Held; duplicates prevented
Service course groups placed atomically in one slot	Hard	Held in all runs
Rest gap of 1-3 days between a cohort's exams	Soft	Satisfied where period length permitted
Unscheduled courses reported with reasons	Reporting	Confirmed under artificially constrained periods

4.5 Objective 3: QR-Code-Based Presence Verification

4.5.1 Implementation

Venue QR codes encode the plain token `VENUE:{venueId}:{examinationSessionId}`; they are generated server-side with the `qrcode` library — individually or in bulk per session — and printed for posting at each venue. Invigilators scan with the device camera through the `html5-qrcode` library. Verification is a two-stage protocol built on the single shared `evaluateVenueScan` function (Figure 7): the `preview` endpoint runs the full validation pipeline and returns the verdict without writing to the database, allowing the invigilator to see the outcome of an accidental or exploratory scan safely; the `confirm`

endpoint re-runs the same pipeline and persists a VenueScan record with the result, the request IP address, the user agent and, where permitted, GPS coordinates checked against the UENR campus bounding box. This venue-bound approach draws on the fraud-mitigation principles identified by Nwabuwe et al. [12], but applies them to invigilator verification without requiring a native mobile app. Crucially, rejected outcomes are persisted as well, so the scan table forms a complete forensic record of every attempt.

Two-Stage QR-Code Verification Flow

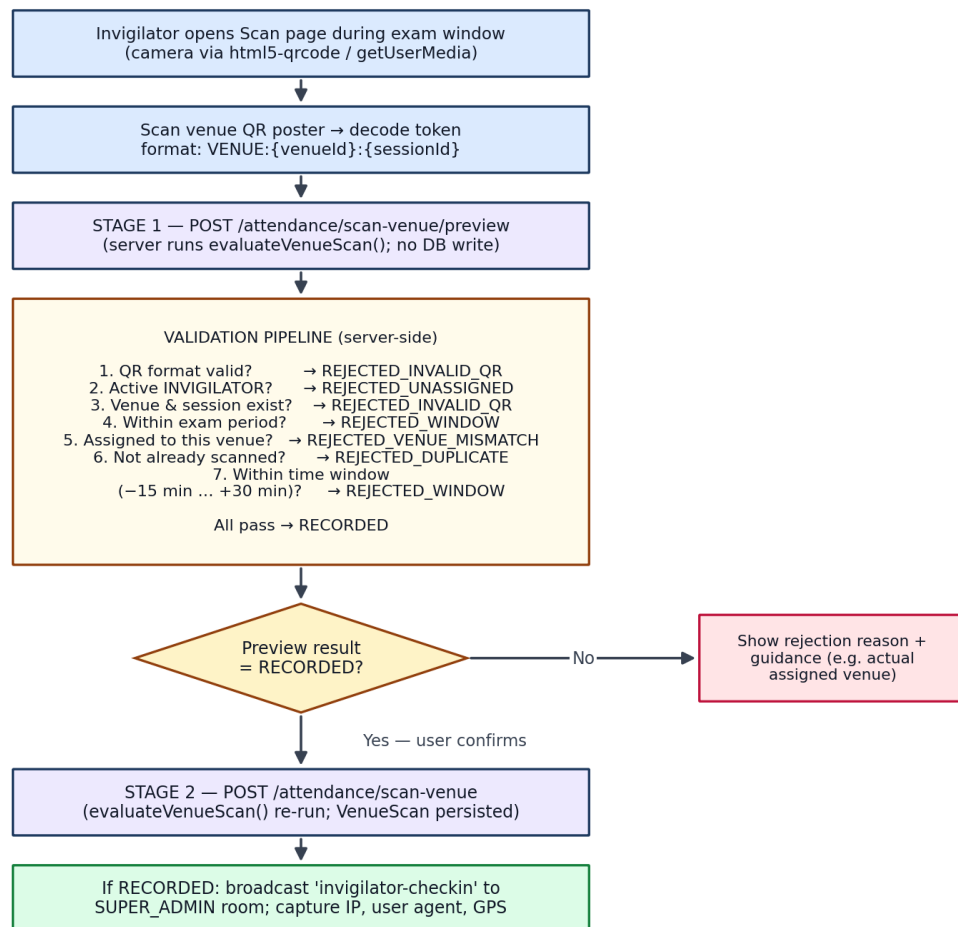


Figure 8: Two-stage QR-code verification flow with the server-side validation pipeline

4.5.2 Results

Table 14 reports the scenario tests executed against the deployed system. Each fraud vector of the paper-based process is closed: remote signing is impossible because the venue's poster must be physically scanned; wrong-venue scans are rejected with guidance naming the invigilator's actual assigned venue; duplicates and out-of-window attempts are rejected and recorded; and post-hoc signing is precluded by the hard close of the window thirty minutes after the slot ends.

Table 14: QR verification scenario test results

Scan scenario	Expected result	Observed result
Valid scan at assigned venue within window	RECORDED; real-time check-in event to officers	As expected; IP, user agent and GPS captured
Scan of a different venue's code	REJECTED_VENUE_MISMATCH with guidance	As expected; rejection persisted
Second scan after a successful one	REJECTED_DUPLICATE	As expected
Scan before window opens (>15 min early)	REJECTED_WINDOW	As expected
Scan after window closes (>30 min after end)	REJECTED_WINDOW	As expected
Malformed or foreign QR content	REJECTED_INVALID_QR	As expected
Scan by a non-invigilator account	REJECTED_UNASSIGNED	As expected (403 at role layer for other roles)
No scan during entire window	ABSENT record created automatically	Created by background job within 5 minutes

4.6 Objective 4: Automatic Invigilator Assignment

4.6.1 Implementation

The venue assignment service implements the round-robin algorithm of Section 3.8.2, shown in Figure 9, following the sequential approach recommended by Cimen et al. [9]. Demand per venue and slot is one invigilator per fifty students, bounded between one and four. The round-robin cursor distributes workload evenly across the active pool while two

conflict rules are enforced for every candidate: no invigilator may hold two assignments in the same slot, and no invigilator may supervise a course offered by their own department. Where no conflict-free candidate remains, the slot is left visibly understaffed for manual resolution rather than ever double-booking. Manual assignment through the interface passes the same checks plus a one-time-frame-per-day rule and the per-venue maximum. Assigned invigilators receive an in-app notification, an e-mail, and a venue-assignment-updated socket event.

Automatic Invigilator Assignment — Round-Robin with Conflict Avoidance

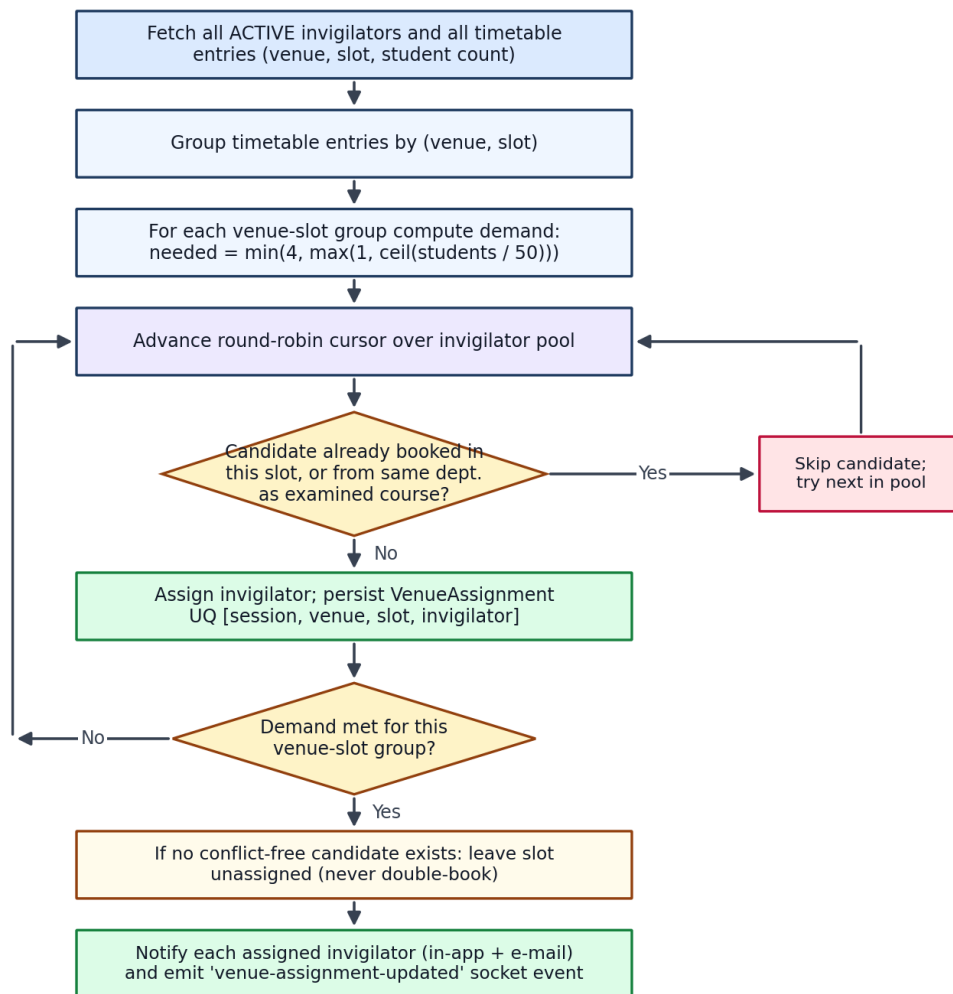


Figure 9: Automatic invigilator assignment algorithm

4.6.2 Results

Inspection of assignments generated over the seeded data confirmed the absence of double-bookings and same-department assignments across all venue-slot groups, with workload spread evenly by the cursor. The database-level unique constraint provides a structural second line of defence against duplicates. Objective 4 is therefore satisfied, with the understaffing report providing the examination officer a clear signal when the invigilator pool is insufficient.

4.7 Objective 5: Real-Time Notification and Automatic Absence Detection

4.7.1 Implementation

The real-time layer (Figure 10) authenticates every Socket.IO connection by verifying the JWT presented in the handshake, loading the user and rejecting non-active accounts; each socket then joins a per-user room and a per-role room. Services publish through a small broadcast utility offering toRoles and toUser primitives. The client registers listeners in the Topbar component, filtered by role so that operational events are handled only by examination officers; handlers raise a toast and invalidate the relevant TanStack Query caches [25], causing affected screens to refetch and re-render immediately. Table 15 lists the event catalogue. Complementing the push path, a background job runs every five minutes, examines venue assignments of the preceding twenty-four hours whose windows have closed, and, for any assignment without a RECORDED scan, creates an ABSENT VenueScan and notifies all Super Admins.

Table 15: Real-time event catalogue

Event	Recipients	Trigger
notification.created	The affected user	Any persisted notification
invigilator-checkin	role:SUPER_ADMIN	Successful (RECORDED) venue scan
pending-account	role:SUPER_ADMIN	New self-registration completing e-mail verification
course-submitted	role:SUPER_ADMIN and the submitting user	Course submitted for approval
course-approved / course-rejected	role:SUPER_ADMIN (and notification to the department)	Approval decision on a course

venue-assignment-updated	The affected invigilator	Assignment created or removed
--------------------------	--------------------------	-------------------------------

Real-Time Notification Architecture (Socket.IO)

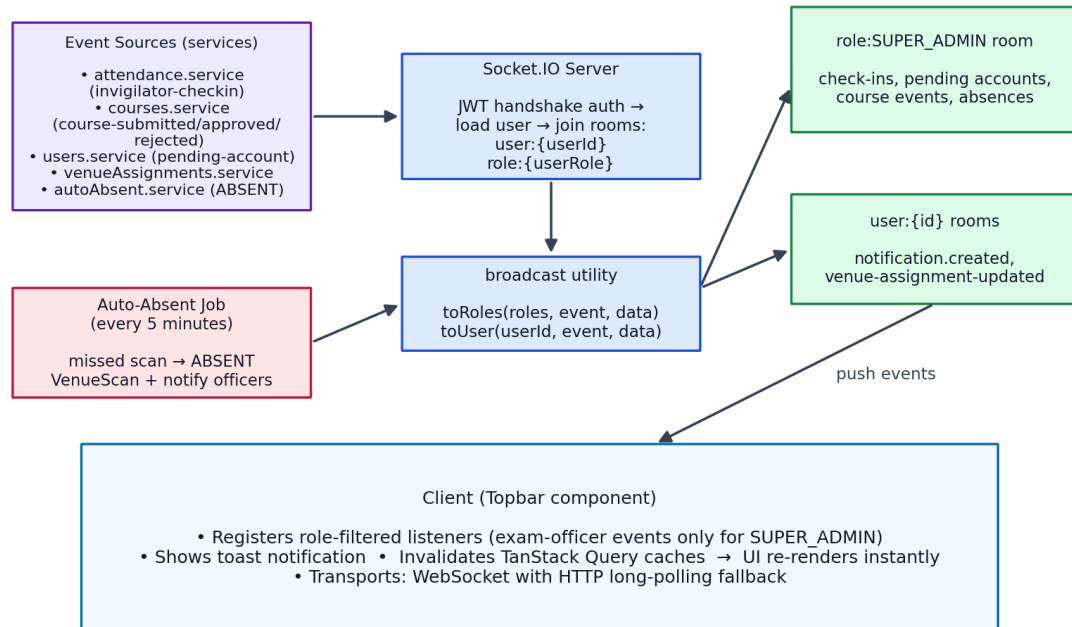


Figure 10: Real-time notification architecture with role-scoped rooms and the auto-absent job

4.7.2 Results

With two browsers connected as different roles, every event in Table 15 was observed to arrive at the correct recipients without a page refresh, and only at those recipients — officers received operational events that department heads and invigilators did not. When a scan window was allowed to lapse, the ABSENT record and the corresponding officer notification appeared within the five-minute job interval. Objective 5 is satisfied: the examination office learns of a no-show within minutes of the window closing rather than days later.

4.8 Objective 6: Role-Based Access Control and Audit Logging

4.8.1 Implementation

Authentication issues a JWT with a one-day expiry on login; the client attaches it as a bearer token to every request and the Axios interceptor redirects to login on any 401

response. The `requireAuth` middleware verifies the signature, loads the user through a sixty-second cache, and rejects accounts that are not `ACTIVE`, so that suspension takes effect within a minute even for holders of valid tokens. Authorisation is declared per route through `requireRole`. A timing-attack mitigation performs a dummy `bcrypt` comparison for sign-in attempts against non-existent accounts, equalising response times [20], [21]. The audit subsystem exposes a single `logAudit` utility invoked by every privileged service action, recording actor, action, target type and identifier, result, JSON metadata, IP address, user agent and timestamp; no update or delete path exists for audit rows, and the actor foreign key is set to null on user deletion so history survives account removal. Nineteen action types are logged, spanning user lifecycle (login, register, approve, reject, update, delete, password change), course workflow (submit, approve, reject), timetable operations (generate, entry update, entry delete, delete), venue assignment (generate, manual assign, remove) and registration windows (set, close).

4.8.2 Results

Cross-role probing of the API — for example, a department head calling the course-approval endpoint or an invigilator calling timetable generation — consistently returned 403 Forbidden, and unauthenticated requests returned 401. Inspection of the audit table after the evaluation sessions showed a complete, chronologically ordered record of every privileged action performed, including the rejected attempts' authentication context. Objective 6 is satisfied at both enforcement layers.

4.9 Security Implementation Summary

Table 16: Security measures by layer

Layer	Measure implemented
Transport	HTTPS termination on all production endpoints (Vercel/Render TLS)
HTTP headers	Helmet-hardened responses; x-powered-by disabled [23]
Cross-origin policy	Strict CORS allow-list bound to the configured client origin
Abuse resistance	express-rate-limit on the API; timing-equalised login comparisons
Credentials	bcrypt hashing with configurable cost [20], [21]; e-mailed reset tokens; 6-digit e-mail verification
Sessions	JWT with one-day expiry; server-side status gate on every request; 401

	interception on the client
Authorisation	RBAC middleware on every route; ProtectedRoute and filtered navigation on the client
Input handling	Zod schema validation of body, params and query on all endpoints [26]
Data access	Prisma parameterised queries throughout (no string-built SQL)
Scan integrity	Venue-bound QR tokens validated exclusively server-side; all outcomes persisted
Account lifecycle	PENDING_APPROVAL gate; SUSPENDED/DISABLED/REJECTED states enforced within 60 s
Location	Optional GPS capture checked against the UENR campus bounding box
Accountability	Immutable AuditLog with actor, action, target, result, IP and user agent

4.10 Discussion

The evaluation shows that all six objectives were met and that the integrated design delivers benefits its parts could not achieve in isolation. Because the timetable generator writes the scan windows that the attendance subsystem later enforces, and because the assignment service feeds both the invigilator's interface and the auto-absent detector, a single chain of custody runs from course approval to the forensic record of each venue scan — exactly the end-to-end accountability that was found missing in the reviewed literature [1], [12].

Three design decisions deserve particular comment. First, binding QR codes to venues rather than users inverts the trust model of prior QR attendance work [10], [11]: the code may be public precisely because possession proves nothing without a valid assignment, an open window and no prior scan, all adjudicated server-side. This aligns with the fraud-mitigation principles of Nwabuwe et al. [12] but extends them to invigilator verification without a native app. Second, the two-stage scan protocol resolves the tension between usability and data hygiene — invigilators can scan exploratively without polluting the record, yet the confirm stage re-validates atomically through the same shared function, so the preview can never promise what the confirmation would refuse. Third, preferring visible understaffing to double-booking in the assignment algorithm accepts a small

manual burden in exchange for a guarantee that the published roster is always physically executable.

The principal trade-off accepted is the greedy scheduler's lack of optimality. For UENR's scale the randomised-restart strategy consistently found complete placements, consistent with the constructive-heuristic results of Abdullah and Hassan [8], but the approach may leave courses unscheduled on substantially larger or more constrained instances where a metaheuristic would succeed [3], [4]; this is mitigated operationally by the actionable unscheduled report and architecturally by the service isolation of the generator, which permits a drop-in replacement. Remaining weaknesses — the absence of an automated test suite, the unimplemented audit viewer, optional rather than enforced GPS, and the lack of formal user-acceptance testing — are honest limitations addressed in the recommendations of Chapter Five.

CHAPTER FIVE: CONCLUSION, RECOMMENDATIONS AND FUTURE WORK

5.1 Conclusion

This project set out to design, build and evaluate an integrated, web-based examination timetabling and invigilator verification system for the University of Energy and Natural Resources. The motivation was simple: the existing manual process was slow, error-prone and open to fraud, and none of the systems reviewed in the literature covered the full lifecycle in one place. The work followed a design science approach [17], [18] and produced a deployed artefact — a React 18 progressive web application [16], [27] over a Node.js/Express REST API with a PostgreSQL database accessed through Prisma ORM — that digitises the entire examination lifecycle from structured course submission to QR-code-based verification of invigilator presence and real-time attendance monitoring.

All six specific objectives were achieved and verified against explicit criteria. A structured course submission and approval workflow replaced e-mail-based collection with a server-enforced state machine, locking of approved courses and full audit logging. A constraint-based greedy first-fit-decreasing timetable generator, with three-pass constraint relaxation and up to twenty randomised retries, produced clash-free timetables with venue splitting for large cohorts and an actionable unscheduled report — results consistent with the constructive-heuristic findings in the recent literature [6], [8]. A venue-bound QR-code verification mechanism, executed through a two-stage preview-then-confirm protocol over a single shared validation function, closed every fraud vector of paper-based attendance while persisting all outcomes for forensic audit — extending the fraud-mitigation principles of Nwabuwe et al. [12] to invigilator verification without a native app. A round-robin invigilator assignment algorithm, following the sequential approach recommended by Cimen et al. [9], enforced no double-booking and no same-department invigilation, preferring visible understaffing to an unexecutable roster. A Socket.IO real-time layer [24] with role-scoped rooms delivered check-in, course and pending-account events without page refreshes, complemented by a five-minute background job that automatically marked missed scans as absent and notified examination officers. Finally, JWT authentication, RBAC middleware and an immutable

audit log established end-to-end accountability across all privileged actions, aligned with OWASP guidance [19], [23].

The evaluation confirms that integrating constraint-based scheduling, venue-bound QR verification and real-time monitoring in a single application is feasible and yields measurable improvements in efficiency, integrity and accountability over the manual baseline. The work also shows that strong physical-presence verification is achievable without biometric hardware by inverting the trust model of conventional QR attendance systems — making the code public and venue-bound, and adjudicating every attempt server-side.

5.2 Recommendations

Based on the findings, the following recommendations are made to the University of Energy and Natural Resources and to the examination office in particular:

28. Deploy the system for a pilot examination period at UENR to evaluate its effectiveness under live operational load and to gather structured feedback from examination officers, heads of department and invigilators.
29. Implement the in-application audit log viewer so that examination officers can review the trail of privileged actions through the interface rather than requiring direct database access.
30. Conduct formal user-acceptance testing with actual UENR staff to validate that the system meets operational requirements and to surface usability improvements before full rollout.
31. Enforce GPS location verification rather than offering it optionally, in order to further strengthen the anti-fraud guarantees of the QR verification subsystem, drawing on the geofencing approach of Nwabuwe et al. [12].
32. Establish an automated test suite (unit, integration and end-to-end) to safeguard the system against regression as features evolve and to support future maintenance.
33. Provision a deployment region closer to Ghana to reduce network latency for end users and to improve the responsiveness of the real-time layer.

5.3 Future Work

Several extensions of the present system are identified for future work, ordered by anticipated impact:

34. Invigilator swap and replacement workflow: allow invigilators to request replacements or swaps through the system, with Super Admin approval. The Invigilation model already carries a replacementId field in anticipation of this feature.
35. Metaheuristic scheduling: replace or augment the greedy generator with simulated annealing, a genetic algorithm or a tabu search post-processor to improve soft-constraint satisfaction and to handle larger, more constrained instances, as surveyed by Siew et al. [1] and Al-Betar et al. [4].
36. Bulk course import: support ingestion of courses from Excel or CSV files to reduce manual data entry for large departments, with the same Zod validation [26] applied to each row.
37. Analytics dashboard: visualise attendance trends, no-show rates, invigilator workload distribution and timetable utilisation to support evidence-based administration.
38. Web push notifications: implement service-worker-based push so that examination officers receive alerts even when the application is not open in a browser tab.
39. Native mobile applications: provide iOS and Android applications for invigilators with native camera integration and offline scan queuing for venues with poor connectivity.
40. Multi-institution support: extend the data model and configuration to support several universities with institution-specific rules and data isolation from a single deployment.
41. Student-facing portal: allow students to view their personal examination timetables and to receive notifications of venue or schedule changes, closing the loop on the cohort-level scheduling approximation.
42. Precise indoor location: explore indoor positioning systems [14], [15] to verify presence at the level of the examination hall rather than the campus bounding box, further tightening the verification guarantee.

5.4 Closing Remarks

Examination integrity underwrites the credibility of every academic qualification a university awards. The manual processes that prevail at many institutions, including UENR, leave that integrity exposed to scheduling error, undetected absenteeism and unaccountable administrative decisions. This project has shown that a single, well-scoped web application — built from mature, freely available technologies and operating on

hardware that staff already carry — can close those exposures end to end. The artefact is deployed, the objectives are met, and the path to a production rollout at UENR is clear.

REFERENCES

1. [https://www.who.int/news-room/fact-sheets/detail/ebola-virus-disease](#)

APPENDIX A: SYSTEM DEPLOYMENT AND ACCESS

The production deployment of the system is publicly reachable at the following endpoint:

<https://unertimetable.vercel.app>

The application programming interface is hosted at:

<https://uenr-timetable-api.onrender.com/api/v1>

Three demonstration accounts are provisioned for evaluation, one per role. Credentials are available from the project supervisor on request. New accounts may self-register only while an examination officer has opened a registration window; accounts remain in the PENDING_APPROVAL state until an officer approves them.

APPENDIX B: SAMPLE VENUE QR TOKEN FORMAT

Venue QR codes encode a plain-text token of the form:

VENUE:{venueId}:{examinationSessionId}

where {venueId} is the database identifier of the venue and {examinationSessionId} is the identifier of the examination session to which the code applies. The token is human-readable for ease of debugging, but its security does not depend on secrecy: possession of a token is useless without a valid assignment, an open scan window and no prior successful scan, all of which are adjudicated server-side by the shared evaluateVenueScan function.